## 2.9 presence

This helper validates that the specified attributes are not empty. It uses the `blank?` method to check if the value is either `nil` or a blank string, that is, a string that is either empty or consists of whitespace.

```
class Person < ActiveRecord::Base
  validates :name, :login, :email, presence: true
end
```

If you want to be sure that an association is present, you'll need to test whether the associated object itself is present, and not the foreign key used to map the association.

```
class LineItem < ActiveRecord::Base
  belongs_to :order
  validates :order, presence: true
end
```

In order to validate associated records whose presence is required, you must specify the `:inverse_of` option for the association:

```
class Order < ActiveRecord::Base
  has_many :line_items, inverse_of: :order
end
```

If you validate the presence of an object associated via a `has_one` or `has_many` relationship, it will check that the object is neither `blank?` nor `marked_for_destruction?`.

Since `false.blank?` is true, if you want to validate the presence of a boolean field you should use one of the following validations:

```
validates :boolean_field_name, presence: true
validates :boolean_field_name, inclusion: { in: [true, false] }
validates :boolean_field_name, exclusion: { in: [nil] }
```

By using one of these validations, you will ensure the value will NOT be `nil` which would result in a `NULL` value in most cases.

## 2.10 absence

This helper validates that the specified attributes are absent. It uses the `present?` method to check if the value is not either nil or a blank string, that is, a string that is either empty or consists of whitespace.

```
class Person < ActiveRecord::Base
  validates :name, :login, :email, absence: true
end
```

If you want to be sure that an association is absent, you'll need to test whether the associated object itself is absent, and not the foreign key used to map the association.

```
class LineItem < ActiveRecord::Base
  belongs_to :order
  validates :order, absence: true
end
```

In order to validate associated records whose absence is required, you must specify the `:inverse_of` option for the association:

```
class Order < ActiveRecord::Base
  has_many :line_items, inverse_of: :order
end
```

If you validate the absence of an object associated via a `has_one` or `has_many` relationship, it will check that the object is neither `present?` nor `marked_for_destruction?`.

Since `false.present?` is false, if you want to validate the absence of a boolean field you should use `validates :field_name, exclusion: { in: [true, false] }`.

The default error message is *"must be blank"*.